Term Test Solutions – Batch 2021-22

Computer Security · Software Project Management · Machine Learning · HCI

- **CSE 461** Introduction to Computer Security (Term test)
- **CSE 421** Software Project Management (Term test 1 & 2)
- **CSE 475** Machine Learning (Mid #1 & Mid #2)
- **CSE 495** Human Computer Interaction (Midterm #01 & #02)

1. Computer Security (CSE 461) – SUST e-certificate system

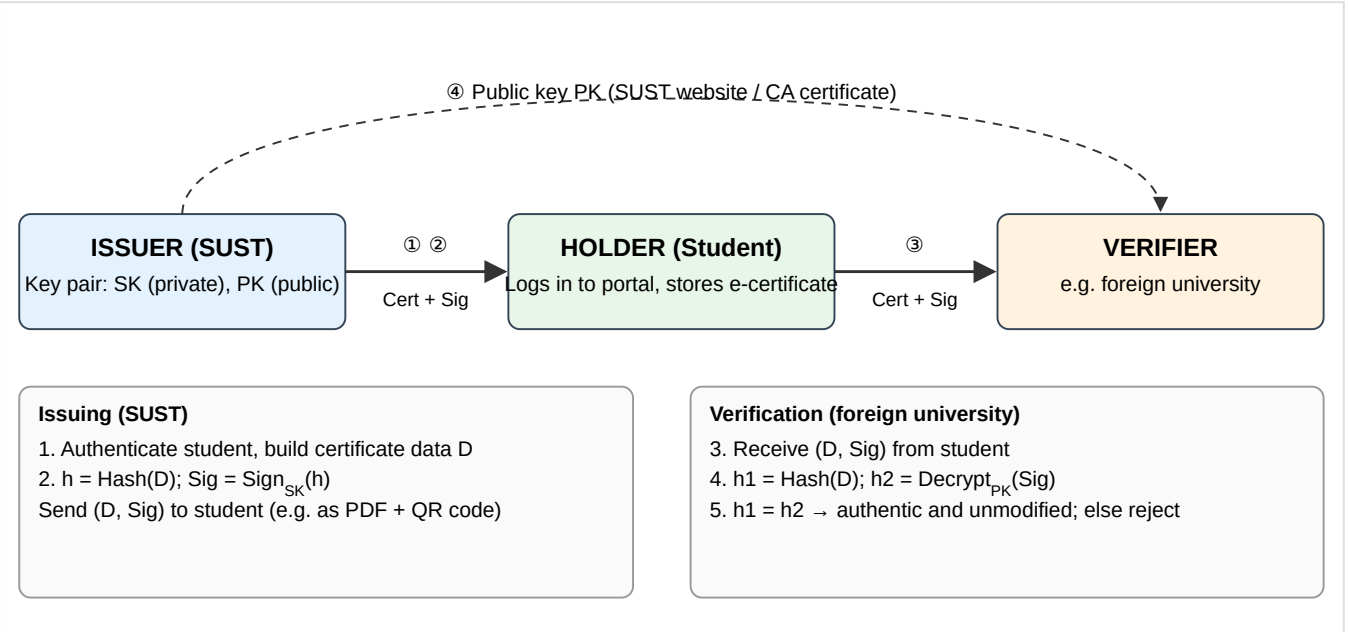
(a) Use of hash (2). Can the system work without hashing?

A hash function (e.g. SHA-256) turns the certificate/result data into a short, fixed-length digest. The issuer **signs this digest** with its RSA private key instead of signing the whole document.

- **Integrity:** any change to the certificate (even one bit) changes the digest, so tampering is detected during verification.
- **Efficiency:** RSA is slow and can only process data smaller than its modulus; signing a small digest is fast for documents of any size.
- **Fixed-size signature**, and hashes are also used to store portal passwords safely (not stored in plain text).

Answer: Without hashing the system could in theory still sign the whole document block-by-block with RSA, but it would be very slow, produce huge signatures and be open to block-reordering/substitution attacks. In practice the system **cannot function properly/securely without hashing**.

(b) System design with Issuer, Holder and Verifier (4)



1. The student logs in to the portal (authentication) and requests an e-certificate; SUST prepares the certificate data D (name, ID, results, date, serial no.).

2. SUST computes h = Hash(D) and signs it with its **private key**: Sig = $\text{Sign}_{SK}(h)$. D + Sig (often with a QR code) is given to the student (holder).
3. The student sends D + Sig to the verifier (e.g. a foreign university).
4. The verifier obtains SUST's **public key** (authentic via a CA-issued certificate or SUST's website), recomputes h1 = Hash(D) and decrypts the signature with PK to get h2.
5. If h1 = h2, the certificate is genuine and unmodified; otherwise it is rejected. The private key never leaves SUST.

(c) Could we use MAC instead of PKI? (2)

No. A MAC is symmetric: the same secret key creates and verifies the tag. Every verifier (any foreign university) would need SUST's secret key, so (i) the key would have to be shared securely with an unlimited number of unknown parties, (ii) any verifier could **forge** a certificate because it holds the same key, and (iii) there is no **non-repudiation** (SUST could deny, or anyone could claim, it issued a certificate). PKI/digital signatures need only a public verification key, which can be published openly. (MAC would only suit a closed, trusted two-party setting, e.g. portal ↔ SUST server integrity.)

(d) Security principles maintained / not maintained (2)

Maintained	Not maintained / weak
• **Integrity** – hash + signature detect modification • **Authenticity** – signature proves SUST issued it • **Non-repudiation** – only SUST holds the private key • **Confidentiality / access control** – a student sees only his own result • **Psychological acceptability** – portal is user-friendly	• **Availability** – no mention of redundancy, backup or DoS protection (single portal) • **Confidentiality of the certificate itself** – a signature does not encrypt; anyone who gets the file can read it • **Economy of mechanism / cost-effectiveness** – a large budget is consumed every year • **Defense in depth / no absolute security** – the claim that "even top hackers can't break through" is unrealistic; key compromise or insider/portal attacks are still possible • **Accountability** – logging/auditing not mentioned

Term Test 2 – ARP spoofing / MITM on café Wi-Fi (10)

Motu (192.168.0.23, MAC …:00:23) is on the café Wi-Fi with router 192.168.0.1 (MAC …:00:01). Attacker Patlu (192.168.0.50, MAC …:00:50) is on the same network. Motu logs into an old portal that hashes his password client-side with unsalted MD5 and sends it over plain HTTP; the server stores the same MD5. Patlu sniffs the login request with Wireshark and captures the hash 5f4dcc3b5aa765d61d8327deb882cf99.

(a) How Patlu positions himself as a man-in-the-middle via ARP cache poisoning (6)

Normally each device learns the other's MAC by ARP request/reply and caches it:

Before the attack	IP	MAC in cache
Motu's ARP cache	192.168.0.1 (router)	AA:AA:AA:00:00:01 (router's real MAC)
Router's ARP cache	192.168.0.23 (Motu)	AA:AA:AA:00:00:23 (Motu's real MAC)

1. Patlu runs an ARP-spoofing tool (e.g. arpspoof/Ettercap) and first enables **IP forwarding** on his own machine, so traffic he intercepts can still be relayed on to the real destination.
2. Patlu sends Motu an unsolicited (gratuitous) ARP reply: *"192.168.0.1 (the router) is at AA:AA:AA:00:00:50"* (Patlu's own MAC).
3. Patlu simultaneously sends the router a forged ARP reply: *"192.168.0.23 (Motu) is at AA:AA:AA:00:00:50"*.

4. Because ARP is stateless and **unauthenticated**, both Motu and the router accept these unsolicited replies and overwrite their ARP caches, without ever having sent a matching request.
5. Patlu repeats this periodically (e.g. every few seconds) to keep overriding the real entries before they are refreshed.

After the attack	IP	MAC in cache
Motu's ARP cache	192.168.0.1 (router)	**AA:AA:AA:00:00:50 (Patlu — wrong!)**
Router's ARP cache	192.168.0.23 (Motu)	**AA:AA:AA:00:00:50 (Patlu — wrong!)**

Now every frame Motu addresses to the router (and every reply the router sends to Motu) is physically delivered to Patlu's NIC first. Patlu captures it in Wireshark, then forwards it unchanged to the real router/Motu, so the connection still works and neither side notices — a classic **ARP cache poisoning / man-in-the-middle attack**.

(b) Why client-side MD5 hashing gives almost no protection here (2)

> Hashing only moves the secret, it doesn't protect the channel. The browser sends the **same, static hash** on every login (no per-session nonce/salt/challenge), so once Patlu's MITM position lets him sniff the plain-HTTP request, the captured hash **is itself a valid credential** — he can simply replay `5f4dcc3b5aa765d61d8327deb882cf99` to the server (pass-the-hash) and log in as Motu without ever learning the real password. And because MD5 is **unsalted**, this exact hash is a well-known value (MD5 of the word "password"), so Patlu can also look it up instantly in rainbow tables / online hash databases and get the plaintext password itself.

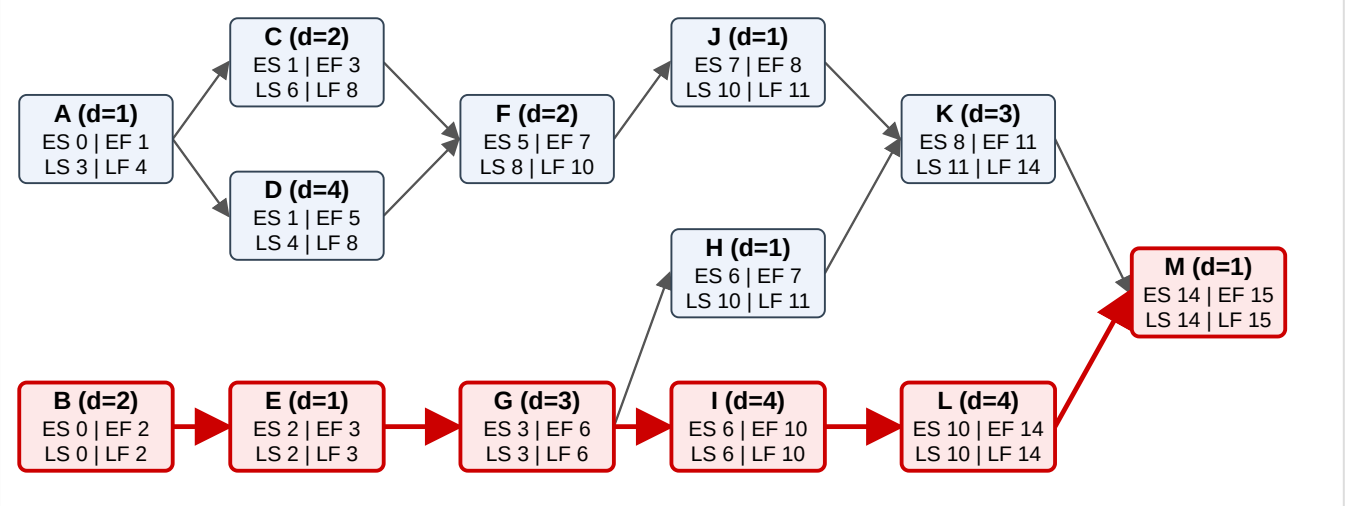
(c) Two countermeasures (2)

- **Encrypt the channel end-to-end with HTTPS/TLS.** Even if Patlu is still a MITM at the network layer, TLS encrypts the login request so sniffing captures only ciphertext, and a valid certificate stops him from silently relaying/altering traffic.
- **Use a salted hash with a per-session challenge/nonce** (e.g., server sends a random nonce, client computes HMAC(password_hash, nonce), or use bcrypt/Argon2 + TLS), so a captured value is single-use and cannot be replayed; salting also defeats precomputed rainbow tables.
- *(Also acceptable: static ARP entries, Dynamic ARP Inspection (DAI)/port security on the switch, or ARP-spoofing detection tools to stop the MITM positioning itself.)*

2. Software Project Management (CSE 421)

Term Test A – Q1: Precedence diagram and CPM (10)

(a) Precedence diagram (activity-on-node; ES/EF from forward pass, LS/LF from backward pass; critical path in red) – 6 marks



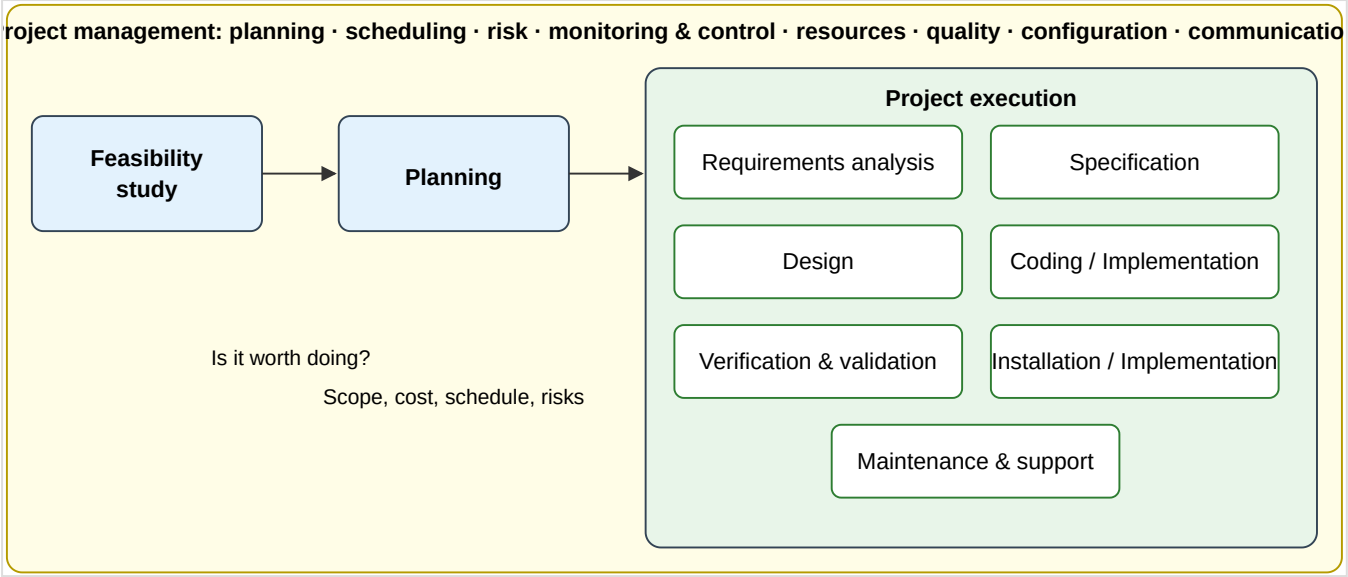
Act.	Dur.	ES	EF	LS	LF	Total float	Critical?
A	1	0	1	3	4	3	
B	2	0	2	0	2	0	**Yes**
C	2	1	3	6	8	5	
D	4	1	5	4	8	3	
E	1	2	3	2	3	0	**Yes**
F	2	5	7	8	10	3	
G	3	3	6	3	6	0	**Yes**
H	1	6	7	10	11	4	
I	4	6	10	6	10	0	**Yes**
J	1	7	8	10	11	3	
K	3	8	11	11	14	3	
L	4	10	14	10	14	0	**Yes**
M	1	14	15	14	15	0	**Yes**

(b) Project duration = 15 weeks.
(c) Critical activities: B, E, G, I, L, **M**.
(d) Critical path: B → E → G → I → L → M = 2+1+3+4+4+1 = 15 weeks (drawn in red above).
(e) Free float of C = earliest start of its successor F − EF of C = 5 − 3 = **2 weeks**.

Check your handwritten answer: (b) 15 and (e) 2 are correct. In (c)/(d) you wrote "B, E, G, I, L" – the last critical activity **M** (it is on the path and has zero float) is missing; also make sure the fourth activity is I (not J – J

has 3 weeks of float).

Term Test B – Q1: Activities covered by software project management (5)



SPM covers the whole life of a software project:

1. **Feasibility study** – is the project technically, economically and operationally worthwhile? (cost-benefit analysis, go/no-go decision)
2. **Planning** – scope, estimation of effort/cost/time, scheduling (CPM/PERT, Gantt), resource and staff allocation, risk planning, quality plan.
3. **Project execution** – requirements analysis, specification, design, coding, verification & validation, installation/implementation, maintenance & support, all of which are monitored and controlled.

Cross-cutting management activities: monitoring & control (progress tracking, corrective action), risk management, quality management, configuration/change management, human-resource and team management, communication and reporting.

Term Test B – Q2: Cost-benefit analysis of four projects (discount rate 10%) (5)

I. Cost-benefit analysis (CBA) compares the expected costs of a project with its expected benefits over time (using net profit, payback period, ROI, NPV, IRR) to judge whether the project is worthwhile and to choose among alternatives.

Project	Total inflow	II. Net profit (NP)	III. Payback (PP)	IV. ROI	V. NPV @10%
P1 (−100,000)	150,000	50,000	4.5 yrs	10%	+620.92
P2 (−1,000,000)	1,100,000	100,000	4.67 yrs	2%	−179,750
P3 (−100,000)	150,000	50,000	3.33 yrs	10%	+13,724
P4 (−120,000)	195,000	75,000	4.0 yrs	12.5%	+21,665

Formulas. NP = total inflows − total costs. Payback = years until cumulative cash flow turns ≥ 0 (with fraction of the last year). ROI = (average annual profit ÷ initial investment) × 100, average annual profit = NP ÷ 5. NPV = $\Sigma\ CF_t/(1.1)^t$. Discount factors: 0.9091, 0.8264, 0.7513, 0.6830, 0.6209 (annuity factor yrs 1-4 = 3.1699; yrs 1-5 = 3.7908).

- **P1:** PP: cumulative 10k, 20k, 30k, 50k after yr 4; need 50k more in yr 5 (100k) → 4 + 0.5. NPV = 9,091 + 8,264 + 7,513 + 13,660 + 62,092 − 100,000 = **+621**.
- **P2:** 4×200k = 800k after yr 4; 200k of 300k needed → 4 + 2/3. NPV = 200,000×3.1699 + 300,000×0.6209 − 1,000,000 = 820,250 − 1,000,000 = **−179,750**.
- **P3:** 30k per year → 100/30 = 3.33 yrs. NPV = 30,000×3.7908 − 100,000 = **+13,724**.

- **P4:** 4×30k = 120k → exactly 4 yrs. NPV = 30,000×3.1699 + 75,000×0.6209 − 120,000 = 141,665 − 120,000 = **+21,665**.

> **Conclusion:** P4 has the highest NPV (and best ROI among the viable ones) → best choice; P3 is second (fastest payback); P1 barely breaks even; P2 has the largest net profit but a negative NPV and a 2% ROI → reject.

3. Machine Learning (CSE 475)

Mid #1

Q1. Learning type for a self-driving car (5)

Primarily supervised learning (deep CNNs/transformers), supported by reinforcement and unsupervised/self-supervised learning.

- **Why supervised:** perception tasks (object/pedestrian/lane/traffic-sign detection, segmentation, steering/behaviour cloning) have a clear ground truth and huge amounts of labelled camera/LiDAR data from human drivers; the model's behaviour is predictable, testable and safety-certifiable.
- **Reinforcement learning** is used for decision-making/planning (lane changes, merging) but mostly in simulation because trial-and-error exploration on real roads is unsafe.
- **Unsupervised/self-supervised** learning is used to pre-train on unlabelled driving video and to find rare scenarios/anomalies.

Q2. Fraud detection – confusion matrix (5)

	Predicted Fraud	Predicted Legitimate	Total
Actual Fraud	TP = 30	FN = 20	50
Actual Legitimate	FP = 70	TN = 9,880	9,950
Total	100	9,900	10,000

- **Accuracy** = (TP+TN)/Total = (30+9880)/10,000 = **99.1%**
- **Precision** = TP/(TP+FP) = 30/100 = **30%**
- Also: Recall = 30/50 = 60%; F1 = 2·0.3·0.6/(0.3+0.6) = 0.40; Specificity = 9880/9950 = 99.3%.

Most appropriate metric: Recall (and F1 / precision-recall trade-off), not accuracy. Only 0.5% of transactions are fraud, so a useless model that labels everything "legitimate" would score 99.5% accuracy – higher than this model's 99.1%. Missing a fraud (FN) is costly, so recall must be high, while precision keeps false alarms (annoyed customers, review cost) in check; hence F1 or F-beta (β>1) and PR-AUC are the right measures.

Q3. Algorithms needing feature scaling; standardization (5)

- **K-Nearest Neighbors** (also K-Means, SVM) – distance-based: a feature with a large numeric range (e.g. salary in thousands vs age) dominates the Euclidean distance, so scaling is required for all features to contribute fairly.
- **Logistic/Linear regression, neural networks (gradient-descent based)** – unscaled features give elongated cost contours, so gradient descent zig-zags and converges slowly or diverges; scaled features give faster and stable convergence. (SVM, PCA are also scale-sensitive.)
- Decision trees split on one feature at a time using thresholds, so monotonic rescaling does not change the splits.

Standardization (z-score normalization): $x' = (x - \mu)/\sigma$, rescaling a feature to mean 0 and standard deviation 1 (μ and σ computed on the training set only and reused on validation/test data).

Q4. 30% of rows have a missing value – remove vs impute (5)

- **Amount of data lost:** deleting 30% of rows discards a lot of information; only acceptable if the dataset is very large.
- **Missingness mechanism:** MCAR (random) – deletion causes little bias; MAR/MNAR – deletion biases the sample, so impute or model the missingness.

- **Missing % per column:** drop a feature that is mostly missing (e.g. >50-60%) and unimportant; impute otherwise.
- **Feature importance and type:** numeric → mean/median (median if skewed/outliers), categorical → mode/"Unknown" category; advanced: KNN or iterative (MICE) imputation.
- **Model used:** some models (XGBoost/LightGBM) handle NaN natively; linear models and neural nets need complete data.
- **Is missingness informative?** add a "was_missing" indicator. Fit imputers on training data only to avoid leakage, and use domain knowledge.

> With 30% of rows affected, imputation is usually preferred over removal.

Q5. Building a high-quality training dataset (5)

1. Define the problem, target variable and success criteria.
2. Collect data from reliable sources, enough in quantity and representative of real-world conditions (all classes, environments, demographics).
3. Label carefully: clear guidelines, multiple annotators, measure inter-annotator agreement, review disputed labels.
4. Clean: remove duplicates and noise, handle missing values and outliers, fix inconsistencies.
5. Preprocess/engineer features: scaling, encoding, resizing, augmentation.
6. Check and fix class imbalance and bias (re-sampling, class weights).
7. Split into train/validation/test (stratified, no leakage or duplicates across splits).
8. Validate quality, version and document the dataset (datasheet); consider privacy/ethics and update it regularly.

Q6. Batch GD vs SGD vs Mini-batch on 10 million images (5)

> **Choose Mini-batch gradient descent** (e.g. batch size 32–256, typically with Adam/momentum).

- **Batch GD:** one weight update needs a pass over all 10M images – too slow and the data does not fit in memory.
- **SGD (1 sample):** cheap and many updates, but very noisy and cannot use GPU parallelism, so wall-clock time is poor.
- **Mini-batch:** fits in memory, vectorised GPU computation, frequent updates, gradient estimate is stable enough, and the small noise helps escape poor local minima/saddle points.

Q7. Underfitting, overfitting and best fit (5)

Plot (left → right)	Boundary	Diagnosis
Left	Smooth curve separating most points	**Best fit** (good generalisation)
Middle	Very wiggly curve hugging every point	**Overfitting** (high variance)
Right	Single straight line, many mis-classified points	**Underfitting** (high bias)

> I read the three plots from the photo (the left plot's curved dashed line, the middle one's squiggle, the right one's straight line); if your original figure is arranged differently, match by boundary shape as above.

- **Reduce underfitting:** use a more complex model (more layers/features/degree), add or engineer better features, reduce regularisation, train longer, tune the learning rate.
- **Reduce overfitting:** more training data/augmentation, regularisation (L1/L2, dropout), early stopping, simpler model or pruning (trees), feature selection, cross-validation, ensembling (bagging).

Mid #2

Q1. Explain the model.fit snapshot (compulsory, 5)

- `model.fit(train_images, train_labels, epochs=5, batch_size=128)` trains for 5 full passes over the training data using mini-batches of 128 samples.

- **469/469** = steps (batches) per epoch. 60,000 MNIST training images ÷ 128 = 468.75 → 469 batches, so the weights are updated 469 times per epoch (≈2,345 updates in total).
- **Time:** ~3-4 ms/step; epoch 1 takes 4 s (includes start-up/graph building), later epochs ~1-2 s.
- **Loss** falls steadily 0.2559 → 0.1030 → 0.0683 → 0.0488 → 0.0371, and **accuracy** rises 92.53% → 96.97% → 97.97% → 98.52% → 98.93%: the model is learning and converging; the gains per epoch shrink (diminishing returns).
- These are **training** metrics only (no validation data was passed), so they say nothing yet about generalisation/overfitting; evaluate on test data.
- `<keras.callbacks.History at 0x7f07...>` is the History object returned by fit(); `history.history` stores the loss/accuracy of each epoch for plotting.

Q2. Steps in training an ANN (5)

1. **Prepare data** and define the architecture (layers, activations), loss function and optimizer.
2. **Initialise weights and biases** (small random values, Xavier/He).
3. **Forward propagation:** pass inputs layer by layer to get predictions ŷ.
4. **Compute the loss** between ŷ and true y (e.g. cross-entropy, MSE).
5. **Backpropagation:** use the chain rule to compute the gradient ∂L/∂w of every weight.
6. **Update the weights** with the optimizer: $w \leftarrow w - \eta \cdot \partial L/\partial w$.
7. **Repeat** for all mini-batches and epochs until the loss converges or early stopping triggers; monitor the validation set.

Q3. Escaping a local minimum (5)

- Use **SGD/mini-batch** (gradient noise can jump out of shallow minima).
- **Momentum** (or Nesterov) to roll over small dips.
- **Adaptive optimizers** – Adam, RMSprop, AdaGrad.
- **Learning-rate schedules**, cyclical learning rates or warm restarts (increase LR to jump out).
- **Random restarts/different weight initialisations**, better initialisation (He/Xavier).
- Batch normalisation, adding small noise, simulated annealing; larger networks (most critical points are saddles rather than bad local minima).

Q4. Mitigating early overfitting (5)

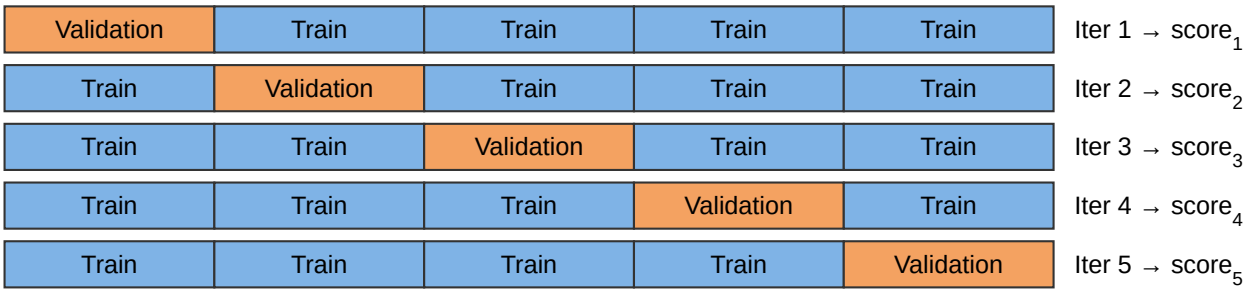
- **Early stopping** when validation loss starts rising.
- **Regularisation:** L2 (weight decay) / L1.
- **Dropout** layers.
- **More data** and **data augmentation**.
- **Simplify the model** (fewer layers/neurons).
- Batch normalisation, cross-validation, reducing training epochs, adding noise.

Q5. k-fold cross-validation vs hold-out (5)

Hold-out (one split)



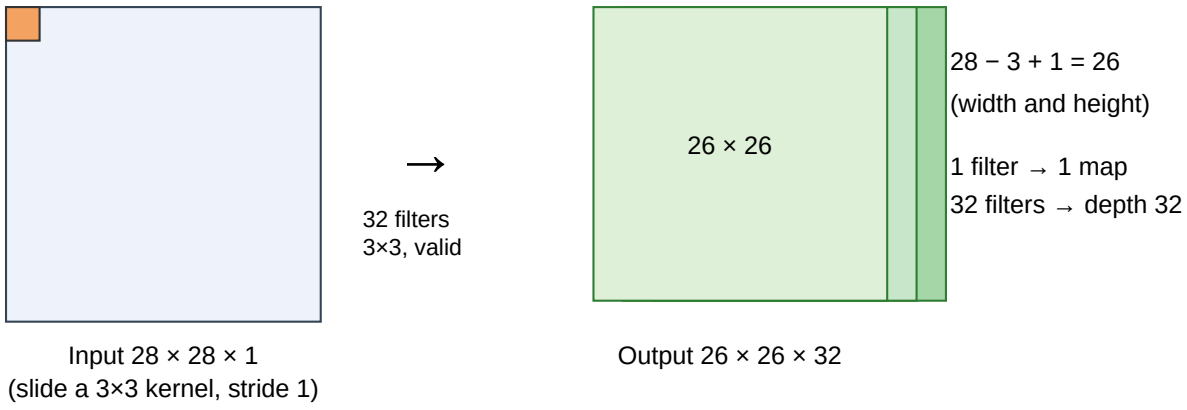
k-fold (k = 5): every fold is the validation set once



Final performance = mean of the 5 scores

	Hold-out	k-fold cross-validation
Split	Data split once (e.g. 80/20)	Data divided into k equal folds
Training runs	1	k (each fold is the validation set once)
Estimate	Depends on one random split; high variance	Average over k splits; more reliable, lower variance
Data use	Part of the data never used for training	Every sample is used for both training and validation
Cost	Cheap, fast	≈ k times more computation

Q6. Output size of the convolution layer (5)



Output size = (N − F + 2P)/S + 1 = (28 − 3 + 0)/1 + 1 = **26** for each spatial dimension (kernel F = 3×3, stride S = 1, no padding, i.e. "valid"). The 3×3 kernel has 26 possible positions horizontally and 26 vertically over the 28×28 image. Each filter produces one 26×26 feature map; the layer has **32 filters**, so depth = 32 → output (26, 26, 32). Parameters = (3×3×1 + 1)×32 = 320. (`Conv2D(32, (3,3), activation="relu")`)

Q7. Optimizer, loss and metric not listed (5)

- **Optimizer:** `"adam"` (others: SGD with momentum, Adagrad).

- **Loss function:** `"mean_absolute_error"` (MAE) – suitable here because Dense(1) is a regression output; other options: Huber loss; for classification: binary/categorical cross-entropy.
- **Evaluation metric:** `"mean_absolute_error"` (or `"mae"`/R^2).

> Note: "accuracy" is not meaningful for a single-neuron regression model; MAE/RMSE are better metrics.

4. Human Computer Interaction (CSE 495)

Midterm #02 (20 marks)

Q1. Assumption vs claim in conceptualizing interaction design

- **Assumption:** something taken for granted about users, their needs or the context that is accepted as true without being proven; it must be checked (e.g. by user research). *Example:* "University students always carry a smartphone and prefer to check their results on it."
- **Claim:** a statement that a particular design feature or product **will have a specific effect** (a hypothesis about the design, with possible upsides and downsides) that is to be tested/evaluated. *Example:* "A one-tap 'Buy now' button will speed up checkout and reduce abandoned carts (but may cause accidental purchases)."

> Assumptions describe the users/context that already exist; claims state the expected consequences of a design decision.

Q2. Shneiderman's three properties of direct manipulation and two benefits

1. **Continuous representation** of the objects and actions of interest.
2. **Physical actions** or button presses (drag, click, touch) instead of complex typed commands/syntax.
3. **Rapid, incremental, reversible actions** whose effect on the object of interest is **immediately visible**.

Benefits: (i) new users learn quickly, and infrequent users remember the interface easily (e.g. drag a file to the trash); (ii) fewer errors and less need for error messages because actions are reversible and feedback is instant, which also reduces anxiety and builds confidence. (Also: experienced users work fast; encourages exploration.)

Q3. Computational offloading; annotation vs cognitive tracing

Computational offloading is using an external tool or representation (pen and paper, calculator, diagram, notes) to reduce the mental effort of a task, so the mind does less computing and remembering. *Example:* doing long multiplication on paper instead of in the head.

Annotation	Cognitive tracing
Modifying an existing external representation by making marks to show what is done or important. *Example:* crossing out bought items on a shopping list; highlighting a textbook.	Externally manipulating items into different orders or structures to keep track of state or to explore options. *Example:* rearranging Scrabble tiles to find a word; moving cards to sort them.

Q4. Gulf of Execution vs Gulf of Evaluation

- **Gulf of Execution:** the gap between what the user wants to do (goal) and the actions the system provides/ makes obvious to do it – "How do I do it?" *Example:* an unlabelled icon, so the user cannot find how to export a file.
- **Gulf of Evaluation:** the gap between the system's output/state and the user's ability to perceive and interpret it to know whether the goal was achieved – "What happened?" *Example:* pressing Save and getting no confirmation.

> **Why bridging matters:** it makes the interface discoverable and understandable (Norman's visibility, feedback, mapping, constraints), cuts errors and frustration, reduces cognitive load and learning time, and increases efficiency and satisfaction, i.e. a better user experience.

Q5. Functional vs non-functional requirements

	Functional	Non-functional

Meaning	What the system must **do** (functions, behaviour, features)	How well it must do it – qualities and constraints
Examples (online shopping app)	User can search products; add to cart; pay online; track an order	Pages load within 2 s (performance); 99.9% uptime (reliability); data encrypted (security); usable by novices; works on Android/iOS (compatibility); accessible

Midterm #01 (CSE 495, Session 2021-22, 20 marks)

Q1. Disabled "Submit" button – two of Norman's principles (5)

- **Constraints:** the disabled button *prevents* an invalid action (submitting an incomplete form), so the user cannot make that error – a logical/forcing constraint.
- **Feedback:** the button changes from greyed-out to enabled only when all required fields are correct, giving continuous information about the form's state and progress, ideally with inline hints on what is missing.
- (Others that also apply: **Visibility** – the grey button signals that submission is currently unavailable; **Affordance/signifier** – a greyed button signals "not clickable".)

Q2. Internal vs external consistency (5)

- **Internal consistency:** the same operations, terms, layout and appearance work the same way *within one* application or system (e.g. all dialogs in one app have OK on the right).
- **External consistency:** the interface follows conventions used *across different* applications, devices and platforms.

> **Example of external inconsistency:** the number keypad of a calculator (7-8-9 on top) versus a telephone/ATM keypad (1-2-3 on top). A person switching between the two must re-learn the mapping, so more digits are mistyped.
> **Effect on cognitive load:** the user cannot reuse existing knowledge (mental model) and has to remember which device uses which layout, concentrate more and correct errors, which increases working-memory load, learning time and mistakes.

Q3. Involving users in design (5)

- **Benefit 1 – Better fit to real needs:** users give first-hand knowledge of tasks, context and expectations, so the product is more usable and relevant and usability problems are found early (cheaper to fix).
- **Benefit 2 – Ownership and acceptance:** users feel involved, have realistic expectations and are more likely to adopt the product.

> **Risk of a full-time user on the design team:** they may gradually lose their "user" perspective (start thinking like a designer), become unrepresentative, bring personal bias, and be a heavy cost/time commitment – so their views may not reflect the wider user population.

Q4. Student portal – Double Diamond (5)

- **Discover step tool: user interviews / questionnaires (or observation/contextual inquiry).** Interview students, teachers and staff and survey a larger group of students about how they use the portal today (registration, results, fees), their frustrations and needs. This widens the team's understanding of the problem rather than jumping to a solution.
- **Define step tool: personas (with affinity diagramming or user-journey map).** Group the interview findings, build personas such as "first-year student who is new to the portal" and "teacher uploading grades", and use them to write a focused problem statement (e.g. "students cannot find their results quickly on mobile"). This narrows the many insights to the key design challenge to be solved in the Develop and Deliver diamond.